Evaluación comparativa de redes de clasificación RGB para el diagnóstico de enfermedades en mora

Proyecto MERCOIA — Informe técnico

Jose Barrios
Universidad Industrial de Santander

Septiembre de 2026

Resumen

Se compararon ocho arquitecturas de clasificación de imágenes RGB, entre redes convolucionales, transformers y modelos fundacionales congelados, para diagnosticar tres estados de la mora (*Rubus glaucus*): sana, antracnosis y moho gris. Cada red se entrenó con tres niveles de aumento de datos sobre una partición estratificada 70/15/15 por clase y agrupada por fruto, lo que da 24 corridas. En el conjunto de prueba (45 imágenes) la exactitud estuvo entre 84.4 % y 100 %, y con el aumento base el promedio fue de 95.8 % de exactitud y 0.936 de F1 macro. EfficientNet-B0 y MobileNetV3-L obtuvieron desempeño de primer nivel con unos 16 MB de pesos y 52 MB de memoria en inferencia, veinte veces menos que ViT-B/16. Estos resultados validan el protocolo de entrenamiento, pero no deben leerse todavía como la exactitud de diagnóstico en campo. Cada clase proviene casi por completo de una sola sesión de captura y el conjunto de prueba es pequeño. Ambas limitaciones se discuten en la Sección 6.

1. Objetivo

El pipeline de MERCOIA segmenta el fruto con SAM 3 y clasifica el recorte para dar un diagnóstico al agricultor a partir de una foto tomada con el celular. Este informe documenta la comparación de clasificadores del estado del arte para esa segunda etapa. Se busca responder tres preguntas: (i) qué arquitectura ofrece el mejor desempeño en el dataset actual de mora; (ii) cuánto aporta el aumento de datos; (iii) qué costo computacional tiene cada red de cara a un despliegue móvil.

2. Datos

2.1. Conjunto de datos

Se usaron los recortes de mora segmentados con SAM 3 que están en el directorio `Algorithm/data`. Hay tres clases: `mora_antracnosis`, `mora_moho_gris` y `mora_sana`. Hay 328 imágenes originales. Además, el directorio contiene copias aumentadas precalculadas en disco (`aug_*`) de las dos clases enfermas, que solo se usan para entrenar.

2.2. Partición

La partición es estratificada por clase: 70 % de los frutos de cada clase van a entrenamiento, 15 % a validación y 15 % a prueba. La unidad de partición es el fruto y no la imagen. El identificador se reconstruye a partir del nombre de archivo, de modo que los recortes de una misma mora no se reparten entre conjuntos. Por eso los porcentajes sobre imágenes son cercanos

a 70/15/15, pero no exactos. Las copias `aug_*` se asignan al conjunto de su original y se descartan de validación y prueba. La semilla es 42. La Tabla 1 muestra los conteos resultantes.

Tabla 1: Imágenes por conjunto y clase (mora, partición 70/15/15 por fruto).

Conjunto	Antracnosis	Moho gris	Sana	Total
Entrenamiento (originales)	67	19	147	233
Entrenamiento (con copias `aug_*`)	402	114	147	663
Validación	6	12	32	50
Prueba	8	4	33	45

3. Metodología

3.1. Arquitecturas

Se evaluaron ocho redes preentrenadas (Tabla 2), de cuatro familias: CNN ligeras pensadas para móvil, una CNN moderna, transformers de visión y un modelo fundacional autosupervisado usado como extractor congelado con sonda lineal [2, 4, 3, 5, 6, 7, 8]. En todas se reemplazó la capa de clasificación por una capa lineal de tres salidas.

Tabla 2: Arquitecturas evaluadas.

Red	Familia	Parám. (M)	Preentrenamiento	Ajuste
EfficientNet-B0	CNN móvil	4.0	ImageNet-1k	completo
MobileNetV3-L	CNN móvil	4.2	ImageNet-1k	completo
EfficientNetV2-S	CNN	20.2	ImageNet-1k	completo
ConvNeXt-T	CNN moderna	27.8	ImageNet-1k	completo
Swin-T	Transformer jerárquico	27.5	ImageNet-1k	completo
ViT-B/16	Transformer	85.8	ImageNet-1k	completo
DINOv2 ViT-S/14	Fundacional	22.1	LVD-142M (autosup.)	sonda lineal
DINOv2 ViT-B/14	Fundacional	86.6	LVD-142M (autosup.)	sonda lineal

3.2. Protocolo de entrenamiento

Todas las redes comparten el mismo protocolo:

- Pérdida de entropía cruzada y optimizador SGD con momento 0.9 y *weight decay* 5×10^{-4}.
- Tasa de aprendizaje inicial de 0.01, reducida por un factor de 0.1 en las épocas 20 y 30.
- 40 épocas, lote de 64 y resolución de 224×224, con precisión mixta (AMP).
- Las sondas lineales de DINOv2 usan tasa de 0.1 y lote de 128.
- Se guarda el checkpoint con mayor exactitud de validación y con él se evalúa la prueba.

El entrenamiento se hizo en una NVIDIA GeForce RTX 4070 de 12 GB, con semilla fija y registro en Weights & Biases (proyecto `MercoIA/clasificacion_frutas`).

3.3. Niveles de aumento de datos

`none` Sin aumento en línea y sin copias `aug_*`. Solo se entrena con las 233 originales.

`base` Recorte aleatorio (escala 0.85–1.0), volteo horizontal y vertical, rotación de ±15° con relleno blanco y variación de color suave (brillo, contraste y saturación de 0.10, tono de 0.02). A esto se suman las copias `aug_*`. La variación de color se mantuvo baja a propósito, porque el color es la señal diagnóstica: gris en moho gris, tejido oscuro en antracnosis.

`fuerte` `base` más TrivialAugmentWide [1], que aplica en cada imagen una transformación aleatoria de catorce posibles, con intensidad aleatoria.

3.4. Métricas y medición de costo

Se reportan la exactitud y el F1 macro. El F1 macro es la métrica principal, porque las clases están desbalanceadas y la de moho gris tiene cuatro imágenes de prueba. El costo en inferencia se midió en la misma GPU con lote 1, entrada de 224×224 y *autocast* fp16. Primero se hicieron 30 pasadas de calentamiento y luego se tomó la mediana de 200 pasadas, sincronizando la GPU en cada una. La memoria es el pico reservado por PyTorch durante la inferencia (`max_memory_allocated`).

4. Resultados

4.1. Desempeño en prueba

La Tabla 3 muestra la exactitud y el F1 macro en prueba de las 24 corridas.

Tabla 3: Exactitud y F1 macro en el conjunto de prueba ($n = 45$) según el nivel de aumento. En negrita, el mejor valor de cada fila.

	`none`		`base`		`fuerte`	
Red	Exact.	F1	Exact.	F1	Exact.	F1
EfficientNet-B0	0.978	0.947	**1.000**	**1.000**	**1.000**	**1.000**
EfficientNetV2-S	0.956	0.920	0.911	0.825	**0.978**	**0.973**
MobileNetV3-L	0.956	0.920	**0.978**	**0.947**	**0.978**	**0.947**
ConvNeXt-T	**0.978**	**0.973**	0.956	0.953	0.889	0.859
Swin-T	0.933	0.894	0.911	0.889	**0.956**	**0.953**
ViT-B/16	**0.978**	**0.947**	0.956	0.920	**0.978**	**0.947**
DINOv2-S (lineal)	0.844	0.683	**1.000**	**1.000**	0.911	0.873
DINOv2-B (lineal)	0.933	0.881	**0.956**	**0.953**	**0.956**	0.910
Promedio	0.944	0.896	**0.958**	**0.936**	0.956	0.933

El aumento `base` sube el promedio 1.4 puntos de exactitud y 4.0 de F1 macro frente a `none`. El nivel `fuerte` no suma nada en promedio. El efecto sí depende de la arquitectura:

- EfficientNetV2-S y Swin-T mejoran con `fuerte`.
- ConvNeXt-T y las dos sondas lineales empeoran con `fuerte`. Una sonda lineal sobre características congeladas no tiene capacidad para absorber deformaciones fuertes de la entrada.
- La sonda de DINOv2-S sin aumento es la peor corrida (F1 de 0.683). Confunde cinco de ocho antracnosis con moho gris, lo que indica que depende de las copias `aug_*` para separar las dos enfermedades.

4.2. Curvas de aprendizaje

La Figura 1 muestra la exactitud de entrenamiento y validación por época. En todas las corridas la exactitud de entrenamiento llega cerca del 100 % en las primeras épocas. La mejor exactitud de validación está entre 0.90 y 1.00 con aumento y entre 0.74 y 0.98 sin él. Después de ese máximo la validación se mantiene estable en casi todas las corridas; la excepción es DINOv2-B con `base`, que llega al máximo en la época 3 y termina en 0.84. Es decir, hay memorización del conjunto de entrenamiento, algo esperable con 233 originales, pero en general no hay sobreentrenamiento que dañe la generalización dentro de la distribución. El mejor checkpoint cae entre las épocas 2 y 16 en 19 de las 24 corridas, así que las 40 épocas son más de las necesarias. La excepción notable es ViT-B/16 con `fuerte`, que siguió mejorando hasta la época 39, lo que es consistente con un aumento que regulariza de verdad.

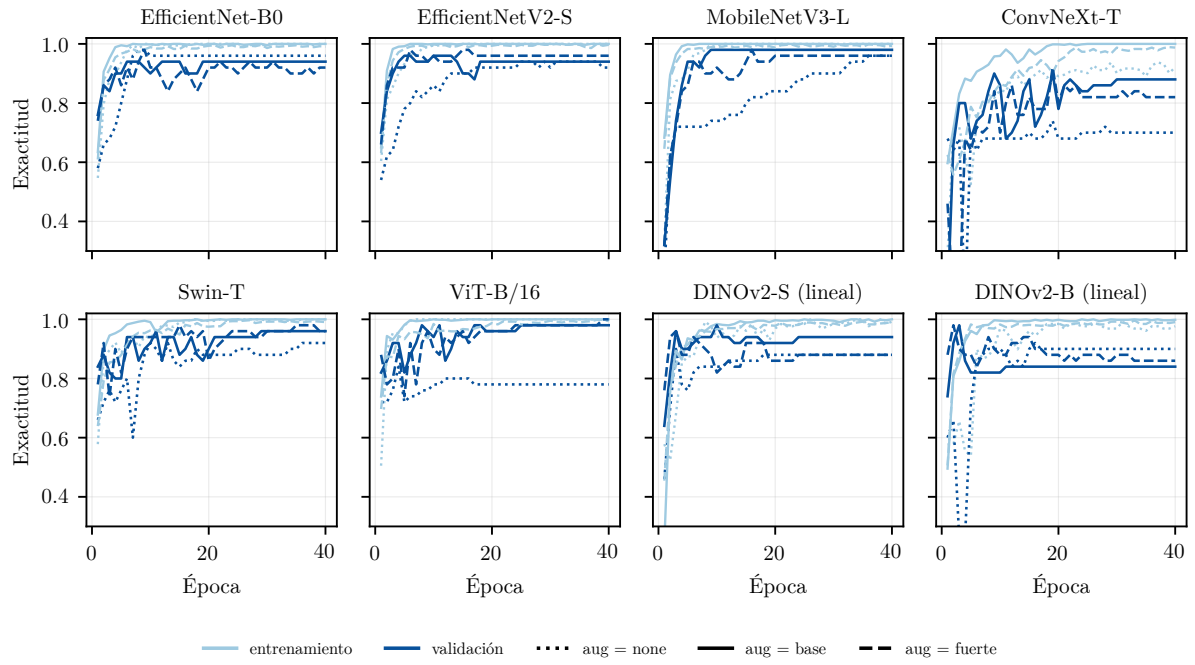


Figura 1: Exactitud por época en entrenamiento (azul claro) y validación (azul oscuro) para los tres niveles de aumento.

4.3. Matrices de confusión

La Figura 2 muestra las matrices de prueba con el aumento `base`. Las matrices de entrenamiento, validación y prueba de las 24 corridas están en el Anexo A. La matriz de entrenamiento se calculó solo sobre las 233 imágenes originales, para que sea comparable con las de validación y prueba.

La mayoría de los errores ocurre en la frontera entre sano y enfermo. Los modelos se reparten en dos grupos con errores de signo opuesto:

- **Falsos negativos.** EfficientNetV2-S, MobileNetV3-L y ViT-B/16 clasifican como sanos algunos frutos enfermos. En la aplicación esto significa no alertar al agricultor.
- **Falsos positivos.** ConvNeXt-T, Swin-T y DINOv2-B clasifican como enfermas algunas moras sanas.

La confusión entre las dos enfermedades es menos frecuente. Aparece sobre todo en la sonda lineal de DINOv2-S sin aumento.

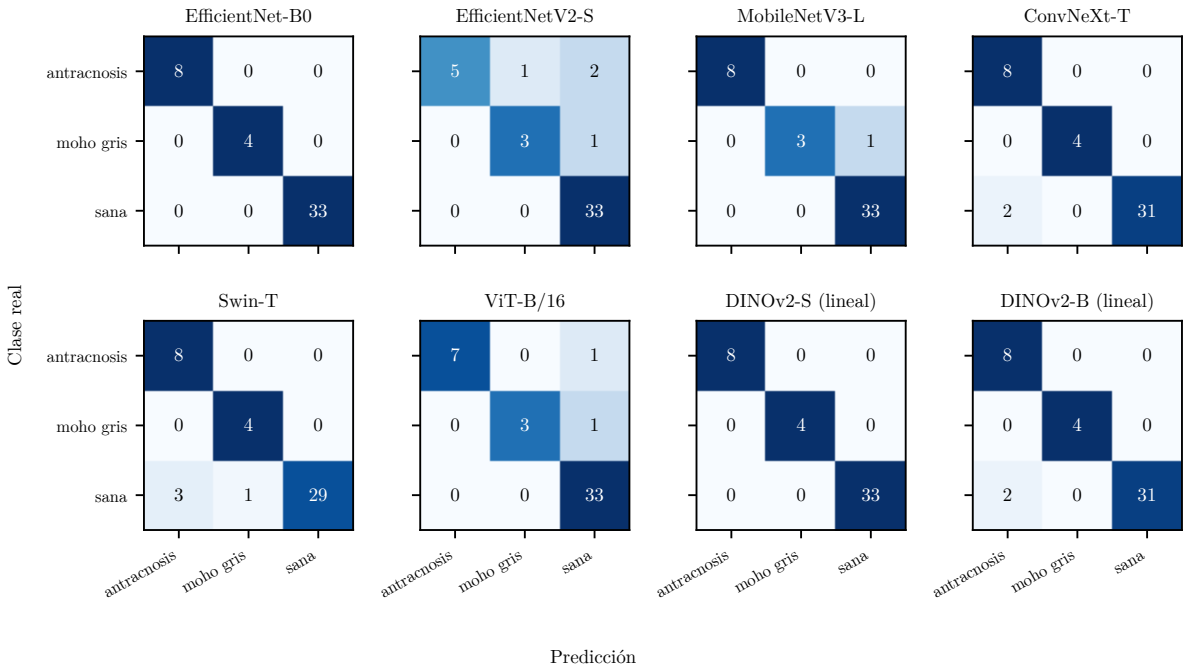


Figura 2: Matrices de confusión en prueba con aumento `base`. El número de cada celda es el conteo de imágenes y el color, la proporción por fila.

4.4. Costo computacional

La Tabla 4 y la Figura 3 resumen el costo de cada red con el aumento `base`.

Tabla 4: Costo computacional con aumento `base` (RTX 4070, lote 1, fp16, 224 × 224). El tiempo de entrenamiento corresponde a 40 épocas.

Red	Parám. (M)	Pesos (MB)	Entren. (min)	Inferencia (ms/img)	Memoria inf. (MB)
EfficientNet-B0	4.0	15.6	2.4	14.2	**52**
MobileNetV3-L	4.2	16.2	2.3	11.0	53
EfficientNetV2-S	20.2	77.8	2.8	30.1	209
DINOv2-S (lineal)	22.1	84.2	2.3	21.6	188
Swin-T	27.5	105.3	2.9	15.1	291
ConvNeXt-T	27.8	106.2	2.7	18.6–45.0[†]	291
DINOv2-B (lineal)	86.6	330.3	2.7	8.4	680
ViT-B/16	85.8	327.4	3.5	**7.2**	848

[†] Rango entre las tres corridas; inestable pese a usar la mediana de 200 pasadas.

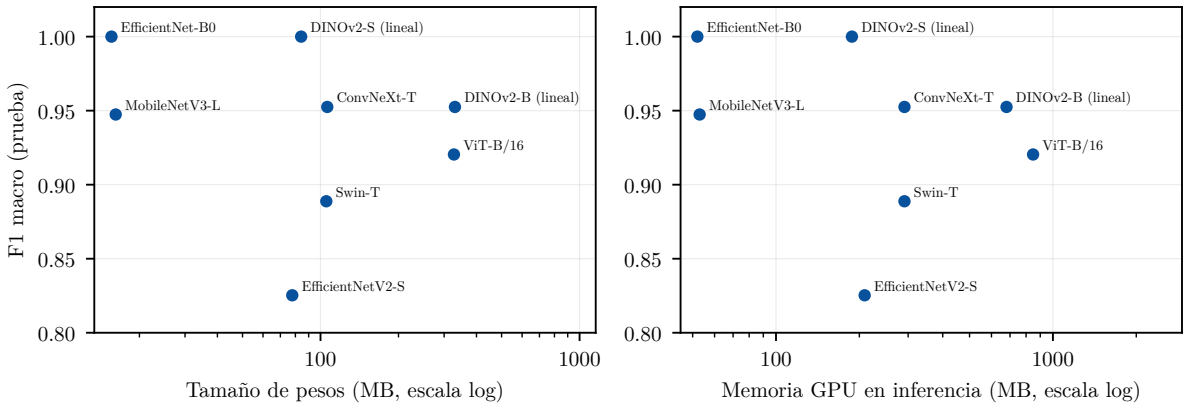


Figura 3: Relación entre F1 macro en prueba y tamaño del modelo (izquierda) o memoria en inferencia (derecha), con aumento `base`.

El tiempo de inferencia en GPU con lote 1 no anticipa el comportamiento en un celular. ViT-B/16 aparece como la más rápida porque, con una sola imagen, el costo en GPU lo domina el número de lanzamientos de kernel y no el número de operaciones. En un dispositivo móvil pesan más el tamaño de los pesos y la memoria. Ahí EfficientNet-B0 y MobileNetV3-L ocupan veinte veces menos que ViT-B/16 y DINOv2-B, con exactitud igual o superior. El entrenamiento es barato para todas las redes: de 2 a 3.5 minutos por corrida, y alrededor de una hora de GPU para las 24.

5. Discusión

Dentro de la distribución del dataset actual, las ocho arquitecturas resuelven la tarea con exactitud igual o superior al 91 % cuando se entrenan con aumento `base`. Las diferencias entre ellas son de una a cuatro imágenes de prueba. Con 45 imágenes, eso no basta para ordenarlas estadísticamente. El intervalo de Clopper–Pearson al 95 % es [0.921, 1.000] para 45/45 aciertos y [0.849, 0.995] para 43/45, así que los intervalos de casi todas las corridas se solapan.

Por eso, el criterio de selección para el prototipo móvil es el costo computacional. EfficientNet-B0 con aumento `base` o `fuerte` es la opción recomendada: logra el F1 más alto de la tabla con la menor huella de memoria. MobileNetV3-L es la alternativa, con el mismo costo y un solo error más en prueba. Los transformers grandes y los modelos fundacionales no dan una ventaja medible en este dataset que justifique su costo.

6. Limitaciones

Confusión entre clase y sesión de captura. Cada clase proviene casi por completo de una sola sesión de captura (Tabla 5). Una red puede separar las clases a partir de la iluminación, la cámara, el fondo o la compresión propios de cada sesión, sin aprender la lesión. Como la prueba sale de esas mismas sesiones, el 95–100 % mide el reconocimiento dentro de la distribución y no la capacidad de diagnosticar moras de otra finca, otro día u otro celular. Esta es la limitación principal del estudio.

Tabla 5: Origen de las imágenes originales de mora por sesión de captura.

Clase	Sesión dominante	Proporción	Resto
Sana	`cap1_13may`	203/212 (96 %)	9 de `cap2_13may`
Antracnosis	lote `Antracnosis`	55/81 (68 %)	14 de `cap5_21jul`, 12 sueltas
Moho gris	imágenes de Internet	24/35 (69 %)	9 de `cap5_21jul`, 2 sueltas

Conjunto de prueba pequeño. La prueba tiene 45 imágenes y solo 4 son de moho gris. Un error en esa clase mueve el F1 macro más de 8 puntos. Todas las corridas usan una sola semilla, así que tampoco se estimó la variabilidad entre repeticiones.

Calidad de las etiquetas. Al cruzar el directorio de entrenamiento con el registro de etiquetado (`etiquetas.csv`) aparecieron tres problemas, contados sobre las tres frutas:

- 398 imágenes no tienen etiqueta trazable.
- 115 imágenes están en una carpeta que no coincide con su etiqueta. Parte son solo cambios de nombre de clase, pero hay contradicciones reales. En mora, 26 recortes etiquetados como sanos y 16 etiquetados como botritis están en la carpeta de antracnosis.
- 1118 moras sanas ya etiquetadas no se incluyeron en el entrenamiento.

Fuentes externas. El 69 % de la clase de moho gris son imágenes de Internet, con un dominio visual distinto al de la captura en campo.

Medición de costo. Los tiempos de inferencia se midieron en una GPU de escritorio. No sustituyen una medición en el dispositivo móvil de destino.

7. Conclusiones y trabajo futuro

1. El protocolo de entrenamiento y evaluación, con partición estratificada por fruto, selección por validación y registro reproducible, funciona correctamente para las ocho arquitecturas.
2. Con el dataset actual, EfficientNet-B0 ofrece el mejor equilibrio entre desempeño (exactitud 1.000 y F1 1.000 en prueba) y costo (15.6 MB de pesos y 52 MB de memoria). Es la red recomendada para el prototipo.
3. El aumento de datos `base` es la configuración por defecto recomendada. El nivel `fuerte` solo conviene en arquitecturas concretas.
4. Antes de reportar una exactitud de diagnóstico en campo hacen falta cuatro pasos: (a) una evaluación con partición por sesión, entrenando sin `cap5_21jul` y probando solo con ella, porque es la única sesión que contiene las dos enfermedades; (b) un conjunto de prueba independiente, de otra finca, otro día y otro dispositivo, con 20 a 30 frutos por clase y reservado exclusivamente para evaluar; (c) la reconstrucción de `Algorithm/data` desde una única fuente de verdad de etiquetas, incluyendo las moras sanas de otras sesiones; (d) la repetición con varias semillas para reportar media y desviación.

Referencias

[1] S. G. Müller y F. Hutter, "TrivialAugment: Tuning-free yet state-of-the-art data augmentation," en *ICCV*, 2021.

[2] M. Tan y Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," en *ICML*, 2019.

[3] M. Tan y Q. V. Le, "EfficientNetV2: Smaller models and faster training," en *ICML*, 2021.

[4] A. Howard *et al.*, "Searching for MobileNetV3," en *ICCV*, 2019.

[5] Z. Liu *et al.*, "A ConvNet for the 2020s," en *CVPR*, 2022.

[6] Z. Liu *et al.*, "Swin Transformer: Hierarchical vision transformer using shifted windows," en *ICCV*, 2021.

[7] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," en *ICLR*, 2021.

[8] M. Oquab *et al.*, "DINOv2: Learning robust visual features without supervision," *TMLR*, 2024.

A. Matrices de confusión por red

En cada figura, las filas corresponden al nivel de aumento (`none`, `base`, `fuerte`) y las columnas, al conjunto (entrenamiento solo con originales, validación y prueba). En todos los casos se usa el mejor checkpoint por exactitud de validación.

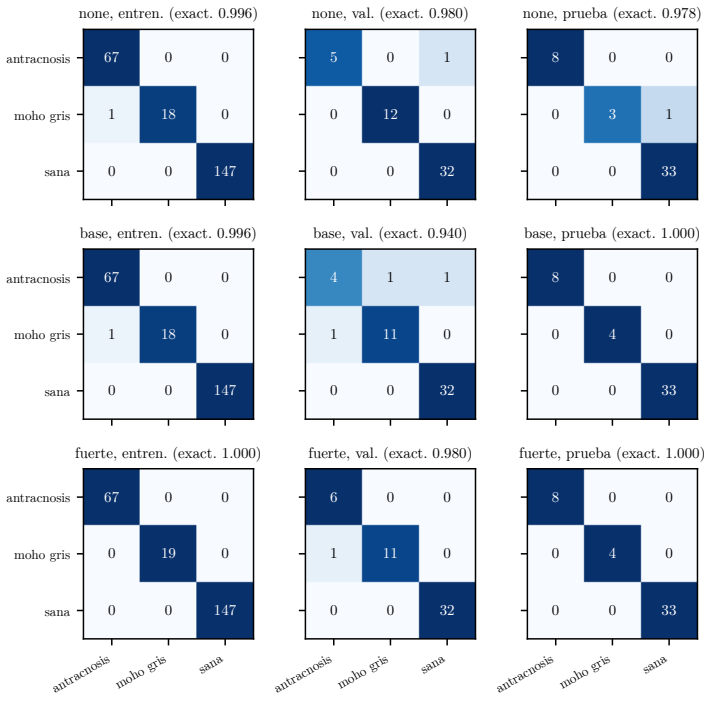


Figura 4: EfficientNet-B0.

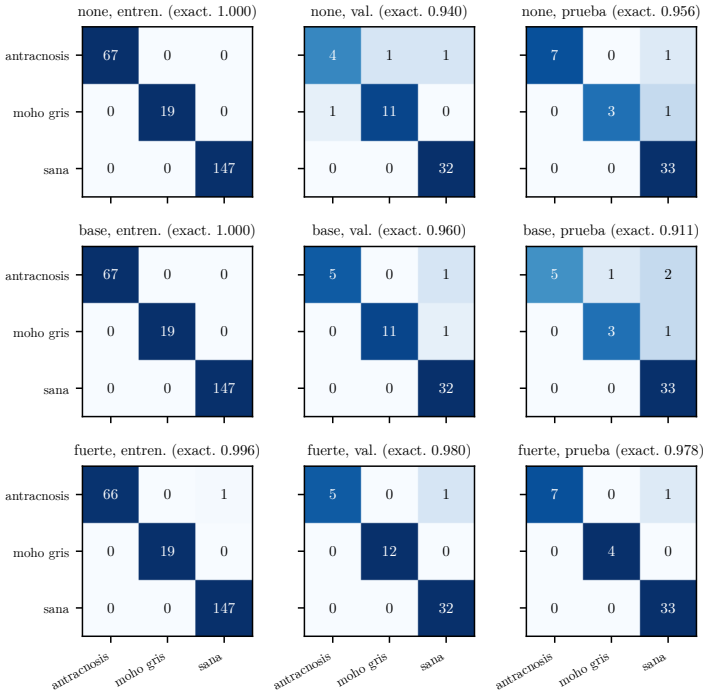


Figura 5: EfficientNetV2-S.

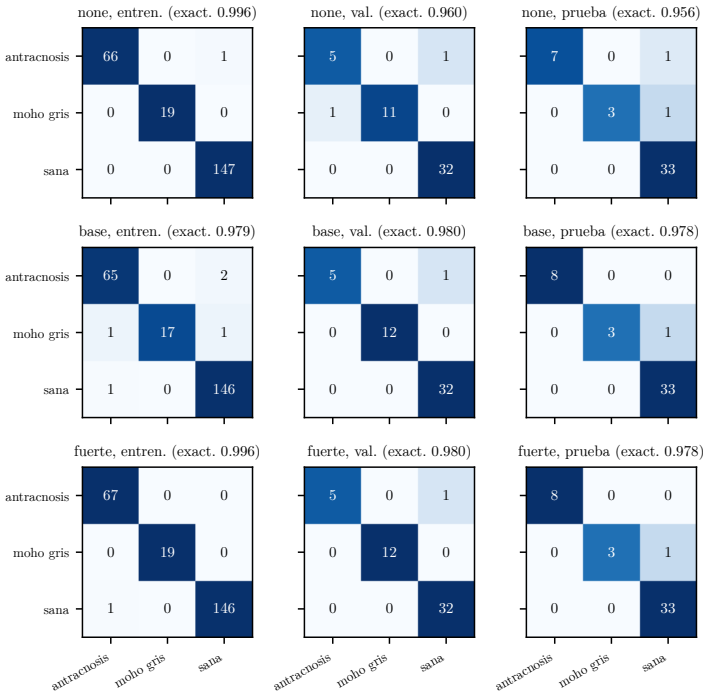


Figura 6: MobileNetV3-L.

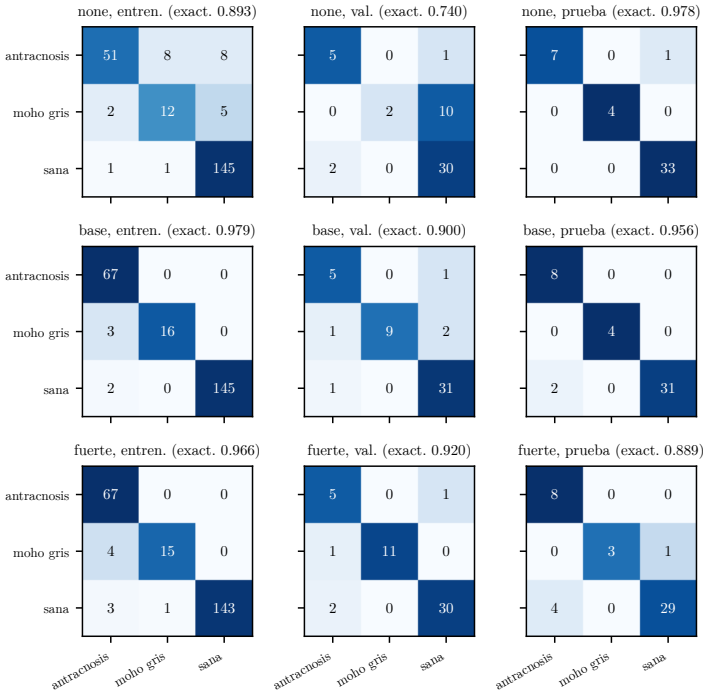


Figura 7: ConvNeXt-T.

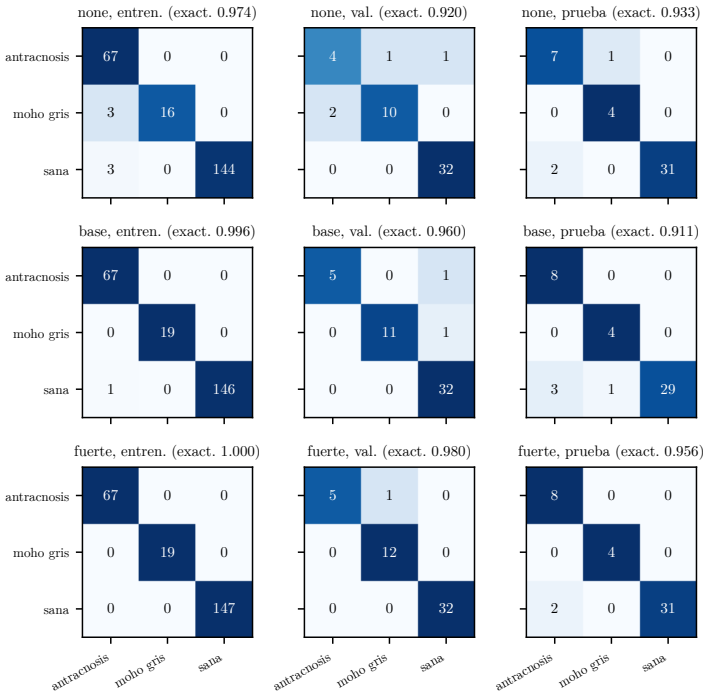


Figura 8: Swin-T.

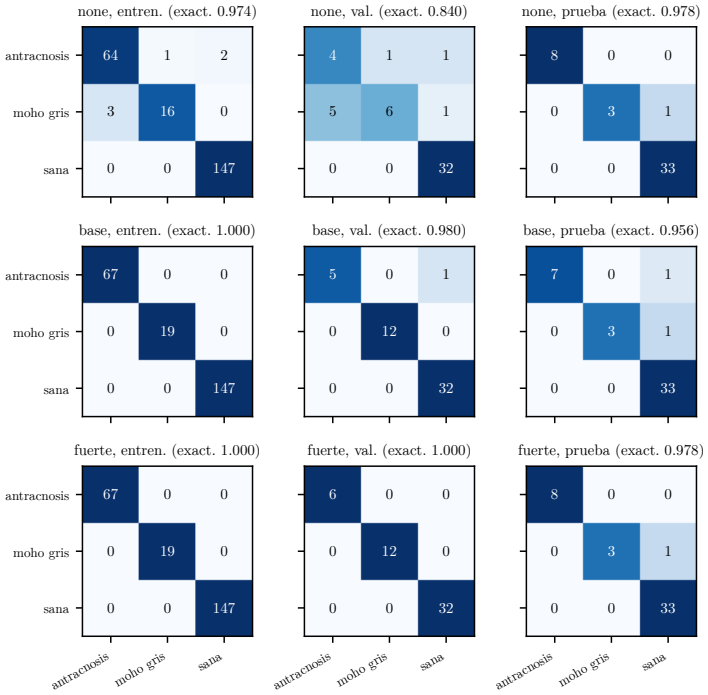


Figura 9: ViT-B/16.

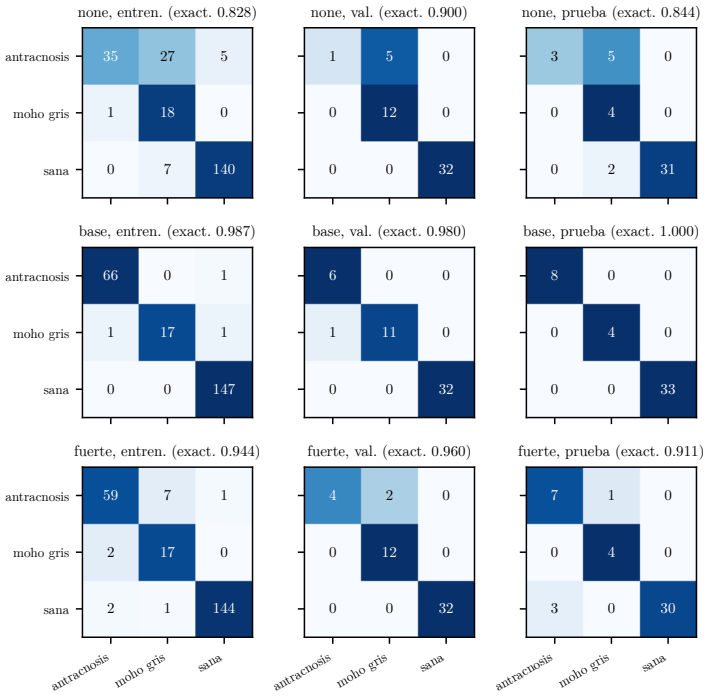


Figura 10: DINOv2 ViT-S/14, sonda lineal.

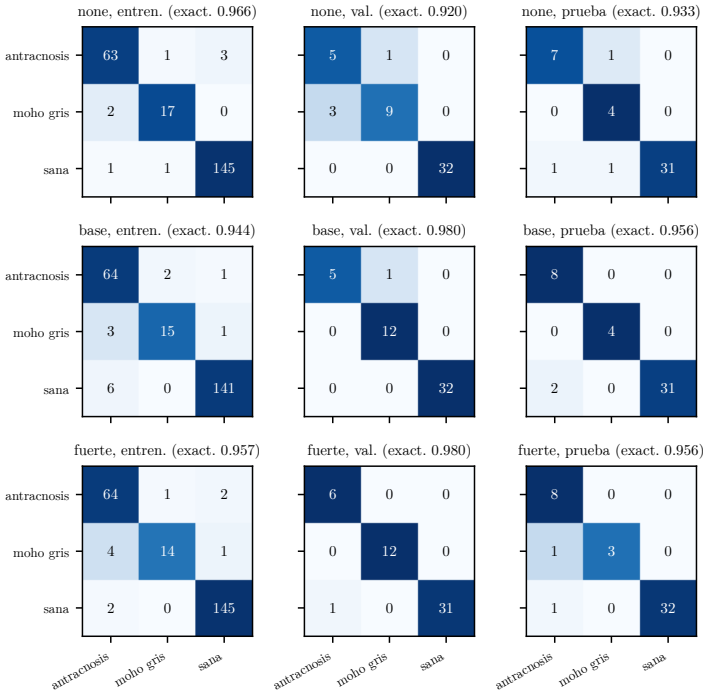


Figura 11: DINOv2 ViT-B/14, sonda lineal.